

MACHINE LEARNING FUNDAMENTALS SERIES

Recurrent Neural Networks (RNNs): Fundamentals & Mechanics

Sequential Data Processing, Unrolling Through Time, and Vanishing/Exploding Gradients

1. Introduction: The Sequential Data Challenge

Standard feedforward neural networks (such as fully connected networks or CNNs) operate under a strict structural limitation: they accept fixed-size input vectors. For instance:

- A single scalar input ($x \in \mathbb{R}$) predicting drug effectiveness.
- A fixed 2-feature vector (x_1, x_2) predicting Iris species.
- A fixed 6×6 grid of pixels for image classification.

However, real-world time-series data—such as stock market prices, audio signals, or natural language sentences—exhibit **variable sequence lengths**. For example, predicting a company's stock price on Day 10 might utilize 9 preceding data points for a mature stock, but only 5 preceding data points for a recently listed stock.

Recurrent Neural Networks (RNNs) overcome this limitation by introducing internal memory via **Feedback Loops**, enabling models to process arbitrary sequence lengths dynamically.

2. RNN Core Mechanics: Feedback Loops & Unrolling

2.1 The Feedback Loop Paradigm

An RNN contains standard weights, biases, and activation functions (e.g., ReLU), augmented by a **Feedback Loop**. At each time step t , the activation output Y_t from the hidden node is fed back into the summation node for the subsequent step $t + 1$.

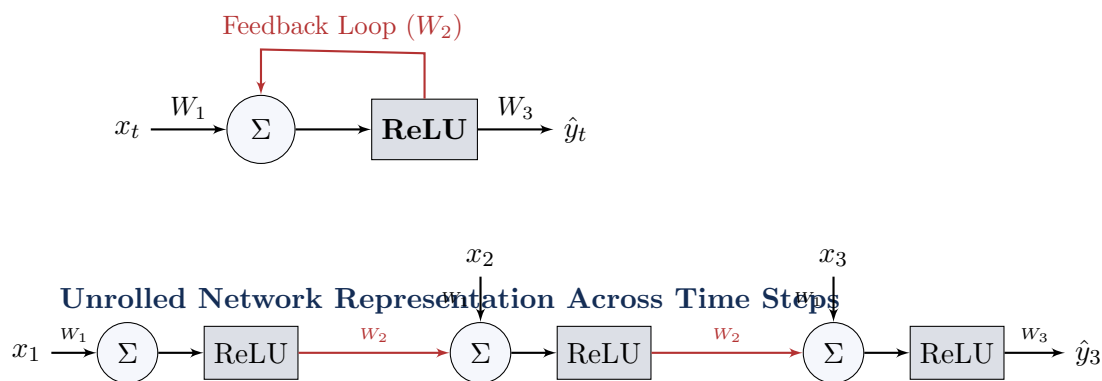


Figure 1: Compact Feedback Loop vs. Unrolled Recurrent Neural Network over 3 time steps.

2.2 Unrolling Through Time & Weight Sharing

To analyze how inputs at earlier time steps influence future predictions, the feedback loop is **unrolled through time**:

1. A structural copy of the base network is created for each sequential input time step $(t_1, t_2, \dots, t_N)$.
2. Input values are fed sequentially from oldest to newest $(x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_N)$.
3. The final output $\hat{y}_N$ combines the current input x_N with historical context propagated through hidden connections (W_2).

Fundamental Principle: Shared Parameters

Regardless of how many time steps an RNN is unrolled (e.g., $N = 2$ or $N = 50$), **the weights** (W_1, W_2, W_3) **and biases** (B_1) **are strictly shared across all time steps**. Unrolling the network increases sequence capacity without increasing the total number of trainable parameters.

3. Worked Example: Stock Market Prediction

Consider predicting tomorrow's stock price based on sequential daily inputs ($x_1 = \text{Yesterday}$, $x_2 = \text{Today}$), scaled such that Low = 0.0, Medium = 0.5, and High = 1.0:

- Step 1 (Yesterday x_1):** Input x_1 enters the network. Hidden node computes $x_{\text{hidden},1} = W_1x_1 + B_1$, passing through ReLU to produce activation $Y_1 = \text{ReLU}(x_{\text{hidden},1})$.
- Step 2 (Today x_2):** Activation Y_1 is multiplied by feedback weight W_2 and added to the new input x_2 .

$$x_{\text{hidden},2} = W_1x_2 + W_2Y_1 + B_1$$
- Output Generation:** The combined activation $Y_2 = \text{ReLU}(x_{\text{hidden},2})$ is multiplied by output weight W_3 to yield tomorrow's predicted price $\hat{y}$.

4. The Vanishing / Exploding Gradient Problem

Despite their architectural elegance, vanilla RNNs struggle when unrolled over long sequences ($N \geq 20$). During **Backpropagation Through Time (BPTT)**, gradients flow backward through N sequence steps, multiplying the feedback weight W_2 repeatedly:

$$\text{Gradient Scale Factor} \propto (W_2)^N$$

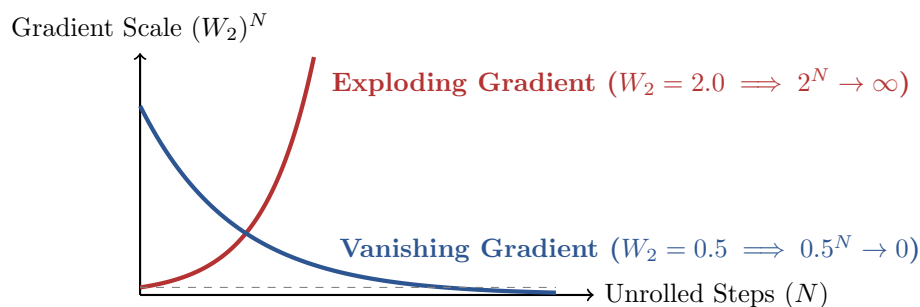


Figure 2: Mathematical behavior of gradient scaling factor $(W_2)^N$ over unrolled sequence length N .

4.1 Exploding Gradients ($W_2 > 1.0$)

When the feedback weight magnitude exceeds 1 (e.g., $W_2 = 2.0$):

- Over $N = 50$ sequence steps, the gradient scaling factor grows as $2^{50} \approx 1.125 \times 10^{15}$.
- Optimization Impact:** The accumulated gradient becomes astronomically large. In Gradient Descent, $\Delta W = -\eta \cdot \text{Gradient}$ yields huge step sizes, causing parameters to overshoot and bounce erratically without reaching loss minima.

4.2 Vanishing Gradients ($W_2 < 1.0$)

When the feedback weight magnitude is less than 1 (e.g., $W_2 = 0.5$):

- Over $N = 50$ sequence steps, the gradient scaling factor shrinks as $0.5^{50} \approx 8.88 \times 10^{-16}$.
- **Optimization Impact:** The gradient vanishes to near-zero ("Hey wait, where the gradient go?"). Step sizes become microscopic, stalling weight updates and preventing the model from learning long-term dependencies.

5. Summary & Modern Evolution

Table 1: Comparison of RNN Gradient Pathologies and Modern Architectural Solutions

Gradient Pathology	Mathematical Cause	Optimization & Practical Consequence
Exploding Gradient	$W_2 > 1.0 \implies (W_2)^N \rightarrow \infty$	Huge step sizes; parameter instability; erratic loss oscillation.
Vanishing Gradient	$W_2 < 1.0 \implies (W_2)^N \rightarrow 0$	Microscopic step sizes; early training stall; inability to capture long-range temporal context.

Stepping Stone to Advanced Architectures

Vanilla Recurrent Neural Networks provide the foundational framework for sequential learning. To overcome the vanishing/exploding gradient bottleneck over long sequences, modern deep learning utilizes advanced gated architectures such as **Long Short-Term Memory Networks (LSTMs)** and **Transformers**.